DSCI 531 Quiz 1

Grammar + channel decisions

Start with the question and observational unit. Grammar: data → statistical transform → mark/geom → channels/aes → scales → guides → facet/layer/layout. A field's storage type does not decide its meaning.

T	Meaning	Implementation
Q	quantitative; differences/ratios meaningful	Altair `"x:Q"`; R numeric continuous scale
N	nominal; unordered identity	Altair `"group:N"`; R character/factor
O	ordinal; ordered, unequal spacing allowed	Altair `"level:O"`; R ordered factor
T	temporal; date/time spacing	Altair `"date:T"`; R Date/POSIXct

Magnitude effectiveness: **common-scale position > unaligned position > length > angle > area > volume/color intensity**. Use x/y for the most important quantities. Identity: spatial separation > hue > shape. Size/area is hard to read precisely; decorative 3D adds distortion/occlusion. Bar/area lengths imply a zero baseline; points/lines may zoom if ticks make it explicit.

- Mapping varies by data: Altair `.encode(color="group:N")`; ggplot `aes(color=group)`.
- Constant styling: Altair `.mark_point(color="steelblue")`; ggplot `geom_point(color="steelblue")` **outside** `aes()`.
- Association ≠ causation. Pooled trend may differ from within-group trends; check grouping/facets. Ecological association does not automatically apply to individuals.

Core chart templates: Altair / ggplot2

```
base = alt.Chart(df)
point = base.mark_point(filled=True).encode(
    x="x:Q", y="y:Q",
    color="group:N", shape="group:N")
facets = point.properties(
    width=130, height=100
).facet("group:N", columns=2)
line = base.mark_line().encode(
    x="time:T", y="value:Q", color="id:N")
line_points = line + line.mark_point(filled=True)
```

```
point <- ggplot(df, aes(x, y, color=group, shape=group)) +
geom_point()
facets <- point + facet_wrap(vars(group), ncol=2)
line <- ggplot(df, aes(time, value, color=id, group=id)) +
geom_line()
line_points <- line + geom_point()
```

- Facet = same specification on subsets; layer = marks share coordinates. Keep scales shared for cross-panel values; free scales emphasize within-group shape but destroy direct magnitude/slope comparison.
- Line requires meaningful order/connection and grouping identity. Interpolated segments are not observations. ggplot `geom_line()` sorts by x; `geom_path()` follows row order. Altair can encode `order=`. Paired data need an ID; never invent pairings.
- Stacked area: top edge = total; each band's value = upper − lower boundary. Use only additive, same-unit, non-overlapping groups. For individual group paths, use lines/facets.

Summaries + raw observations

Mean uses all values and is sensitive to extremes; median is robust but may ignore consequential extremes. SD describes observation spread, **not uncertainty in a mean**. Boxplot: box Q1–Q3, line median, whiskers to most extreme points within 1.5 IQR; beyond-whisker points are not automatically errors.

```
base = alt.Chart(df).encode(
    y="group:N", x="value:Q")
mean_sd = (
    base.mark_circle(color="black").encode(
        x="mean(value):Q")
    + base.mark_errorbar(extent="stdev")
)
box = base.mark_boxplot()
```

```
ggplot(df,aes(group,value)) +
  geom_boxplot()
ggplot(df,aes(y=group,x=value)) +
  geom_jitter(width=0,height=.1,alpha=.3) +
  geom_pointrange(
    stat="summary", fun.data=mean_sdl,
    fun.args=list(mult=1))
```

Always label error bars: raw SD, percentile range, SE, and CI answer different questions. Summary charts can hide multimodality, outliers, or group size; inspect points/rug/distribution.

Counts, proportions, distributions

Count what? Rows may be people, days, or exploded memberships. **Marginal count** = row/column total; **joint count** = one cell/combination, and margins alone do not determine cells. Define denominator before "proportion": joint proportion = cell / grand total; conditional proportion = cell / row or column total.

```
alt.Chart(df).mark_bar().encode(
    x="count():Q",
    y=alt.Y("cat:N").sort("-x"))
# normalized composition
alt.Chart(df).mark_bar().encode(
    x=alt.X("count():Q").stack("normalize"),
    y="group:N", color="cat:N")
```

```
ggplot(df,aes(y=cat)) + geom_bar()
ggplot(df,aes(y=group,fill=cat)) +
geom_bar(position="fill")
# already summarized counts: geom_col(aes(x=n,y=cat))
```

- Bars start at 0. Stacked bars show totals/composition but middle segments lack common baselines; dodge for precise subgroup comparison. Normalized bars hide group size. Preserve natural category order; otherwise sort for comparison. Pie angles/areas are usually harder than bars.
- Histogram bins a quantitative axis and counts observations. Try meaningful bin widths **and origins**; one binning can create/hide peaks.

```
hist = alt.Chart(df).mark_bar().encode(
    x=alt.X("value:Q").bin(step=5),
    y="count():Q")
rug = alt.Chart(df,height=20).mark_tick().encode(
    x="value:Q")
(hist & rug).resolve_scale(x="shared")
```

```
ggplot(df,aes(value)) +
  geom_histogram(
    binwidth=5,boundary=0,color="white") +
  geom_rug()
```

Normalized KDE has total area 1; height is probability per x-unit, not point probability/count. **Count-scaled KDE = n × density:** total area = n; interval area estimates count, but height remains count per x-unit—not histogram-bin count. Small bandwidth = detail/noise; large = smooth/hidden modes. KDE may extend outside valid bounds and can look smooth with tiny n: show rug/n. Separately normalized group densities each integrate to 1, so peak height does not show group size. Violin = mirrored KDE; state scaling/bandwidth and add observations/summary.

```
kde = alt.Chart(df).transform_density(
    "value",
    as_=["value","density"],
    bandwidth=2)
kde.mark_area().encode(
    x="value:Q", y="density:Q")
```

```
ggplot(df,aes(value,color=group)) +
  geom_density(bw="SJ") +
  geom_rug()
```

Joint distributions + EDA

```
# categorical combinations
alt.Chart(df).mark_circle().encode(
    x="a:N", y="b:N",
    size="count():Q", color="count():Q")
# saturated numerical scatter → 2-D histogram
alt.Chart(df).mark_rect().encode(
    x=alt.X("x:Q").bin(step=.1),
    y=alt.Y("y:Q").bin(step=5),
    color="count():Q")
```

```
df |> count(a,b) |> ggplot(aes(a,b,size=n,color=n)) +
geom_point()
ggplot(df,aes(x,y)) + geom_bin_2d(binwidth=c(.1,5))
```

EDA cycle: **expectation → inspect → surprise/agreement → competing explanations → discriminating check → narrow conclusion + unresolved items.**

1. Define row/unit, columns, units, provenance, collection time; check duplicate/missing IDs/dates—not just row count.
2. Overview quantitative distributions, categorical counts, pairwise relationships, and missingness amount/pattern.
3. Valid zero ≠ missing. NA may mean failure/not asked/not applicable/join miss; sentinels like 999 escape `isna()`.
4. Extreme/bimodal/unexpected value: inspect record, related variables, subgroups, provenance, and multiple bin/bandwidth choices; never delete because a plot looks nicer.
5. Pearson = linear; Spearman = monotonic ranks. Near-zero correlation can hide U-shapes/interactions; neither proves causality. Pairwise complete correlations may use different rows.
6. For ML, perform choice-driving EDA on training data; require prediction-time availability. Record what remains unresolved and whether external information is required.